

The Alberta Buck - Ethereum Implementation (DRAFT v0.3)

Perry Kundert

2026-04-20 12:00:00



This document presents a concrete Ethereum implementation of the Alberta Buck Architecture, specifying the five core smart contracts:

1. **BUCK_CREDIT** (ERC-721): An NFT representing an insurer's offer of parametric insurance on a real-world asset, with deterministic depreciation curves and piecemeal client activation.
2. **IdentityRegistry** (custom): An on-chain registry of per-address Pointcheval-Sanders signature credentials and ElGamal-encrypted identity points, verified via a BN254-precompile NIZK at registration time. This is the trust anchor every BUCK transfer is gated on – see the companion Identity and Proofs documents for the full cryptographic design.
3. **BUCK** (ERC-20): A fungible token minted against aggregated BUCK_CREDIT values, with on-chain credit-limit enforcement, default insurance premiums, and **identity-aware approve** and **transfer** flows that enforce the BUCK mutual-decryptability invariant on every counterparty pair via Chaum-Pedersen NIZK re-encryption proofs.
4. **Notes** (custom): A privacy-preserving denomination layer on top of BUCK. Clients escrow BUCK into a commitment pool by posting a Groth16 SNARK that ties the deposited **totalFace** to a batch of Poseidon-hash note commitments; no information about individual note values, flavours, or identity bindings leaves the wallet. See Notes and Proofs for the circuit design.
5. **BUCK_K** (custom): A controller that supplies the dynamic Value Stabilization Factor used in credit-limit computation. The first deployment uses **BuckKControllerStatic**, a governance-settable constant, so the identity and notes layers can land first; the eventual on-chain PID + commodity-basket oracle implementation (**BuckKController**) is fully specified below and slots in at the same interface.

This is the first of several planned implementation approaches. The ERC-721 approach is chosen for its clarity, wide tooling support, and natural fit for unique insured assets. Subsequent documents will examine ERC-1155 (semi-fungible batch credits), ERC-3643 (regulated security token with identity compliance), and ERC-4626 (vault-based credit pooling). The **Implementation Plan** at the end of this document specifies the concrete phased build order and tracks completion status.

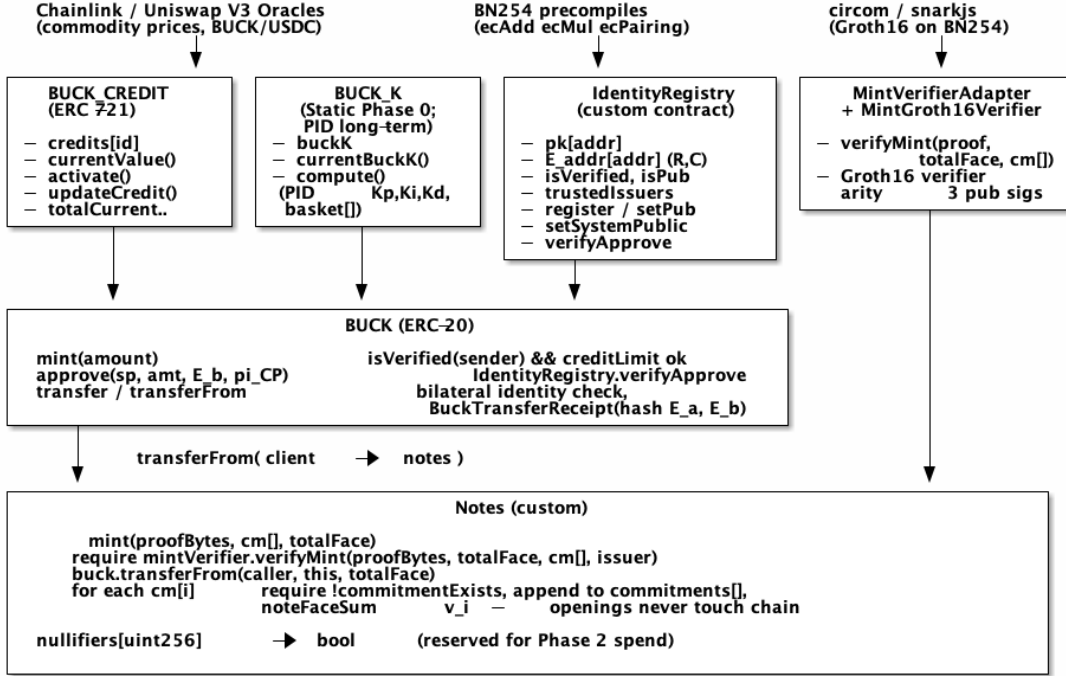
Contents

1	Contract Architecture Overview	5
2	ERC Standard Analysis for BUCK_CREDIT	6
2.1	Recommendation	6
3	BUCK_CREDIT: ERC-721 Insured Asset NFT	7
3.1	Data Model	7
3.2	Depreciation Models	8
3.2.1	Depreciation Examples	9
3.3	Activation (Execution) Model	9
3.4	Insurer Updates	10
4	IdentityRegistry: On-Chain Trust Anchor	10
4.1	Cryptographic Surface	11
4.2	Solidity Surface	11
4.3	Public-Claim Accounts	14
4.4	Issuer Trust	14
5	BUCK: ERC-20 Token	14
5.1	mint() Implementation	14
5.2	Identity-Aware approve()	16
5.3	Identity-Aware transfer() and transferFrom()	17
5.4	Default Insurance Premium	18
6	BUCK_K: Value Stabilization Controller	18
6.1	Process Variable and Setpoint	19
6.2	Solidity Implementation	19
6.3	PID Tuning Considerations	22
6.4	Adaptive Gain Scheduling (Future Enhancement)	22
7	Notes: Privacy-Preserving Denomination	22
7.1	Mint Circuit (circom, Groth16 on BN254)	23
7.2	IMintVerifier and MintVerifierAdapter	23
7.3	Notes Contract	24
8	Integration: Full mint() and transfer() Walkthrough	25
8.1	Step 0: Identity Registration (One-Time)	25
8.2	Step 1: Mint	26
8.3	Step 2: Approve Bob (Identity Handshake)	26
8.4	Step 3: Bob Pulls the Transfer	27
8.5	Step 4: Alice Denominates 125 BUCK as Notes	27
9	Gas Cost Analysis	27

10 Security Considerations	28
10.1 Oracle Manipulation	28
10.2 Credit Enumeration DoS	28
10.3 Insurer Trust	28
10.4 Reentrancy	29
11 Future Approaches	29
11.1 ERC-1155: Semi-Fungible Batch Credits	29
11.2 ERC-3643 (T-REX): Regulated Security Token	29
11.3 ERC-4626: Vault-Based Insurance Pool	29
11.4 L2 and Cross-Chain Deployment	30
12 Implementation Plan	30
12.1 Phase Completion Status (current)	30
12.2 Testing Strategy: Three Layers	30
12.3 Off-Chain Wallet (alberta_buck.wallet, Python)	31
12.4 Phase 0: Stub the Surrounding Contracts	31
12.5 Phase 1: BN254 Helper Library	32
12.6 Phase 2: IdentityRegistry Contract	32
12.7 Phase 3: Identity-Aware Buck ERC-20	33
12.8 Phase 4: Public-Claim Account Support	33
12.9 Phase 5: Notes Scaffolding	33
12.10Phase 6: Groth16 Mint Circuit + Verifier	34
12.11Phase 7 and Beyond	35
12.12Sequencing and Stop Conditions	35

1 Contract Architecture Overview

The Alberta Buck Ethereum implementation comprises five interacting contracts. The `IdentityRegistry` is the trust anchor of every BUCK transfer: `mint`, `approve` and `transfer` are all gated on the sender (and, where relevant, the recipient) holding a verified, registered identity credential. The `Notes` contract sits on top of BUCK as a public (`setSystemPublic`) counterparty whose commitments pool is extended only when the caller supplies a Groth16 SNARK binding the deposited `totalFace` to a batch of Poseidon note commitments.



The flow is:

1. An **insurer** creates a BUCK_CREDIT NFT representing their offer to insure a client's asset.
2. The **client** (asset owner) activates all or part of their credit, paying the current premium.
3. Before the client can interact with BUCK at all, they **register** a Pointcheval-Sanders identity credential and ElGamal-encrypted identity point with `IdentityRegistry`. The contract verifies the issuer's PS signature and the binding NIZK proof on-chain via the BN254 pairing precompile.
4. When the client calls `BUCK.mint()`, the contract requires `IdentityRegistry.isVerified(sender)`, then aggregates the client's active credits, applies depreciation, multiplies by the current BUCK_K, and mints BUCKs up to the resulting limit.
5. To send BUCKs, the sender first calls `BUCK.approve(spender, amount, E_bob, pi_CP)`, which routes the recipient's re-encrypted identity ciphertext and Chaum-Pedersen proof through the `IdentityRegistry`; the proof binds `E_bob` to the sender's registered credential without ever revealing the underlying identity point M .

6. `transfer` / `transferFrom` then perform a bilateral identity check (public-claim recipients skip the receipt requirement, private recipients must have a stored receipt fragment from a prior `approve`) and emit a `BuckTransferReceipt` event carrying only ciphertext hashes.
7. To denominate holdings as private notes, the client calls `Notes.mint(proofBytes, cm[], totalFace)`. The `MintVerifierAdapter` decodes the proof as `(uint256[2], uint256[2][2], uint256[2])` and forwards to the snarkjs-generated `MintGroth16Verifier`, which checks a Groth16 proof with public signals `[totalFace, cm[0], cm[1]]`. The circuit enforces `totalFace =  $\sum v_i$` and `cm_i = Poseidon(flavor_i, v_i, rho_i, idHash_i, predicate_i)` per note. The `Notes` contract is registered `setSystemPublic(true)` so its `transferFrom` pull from the minter satisfies the identity-aware BUCK transfer rules without itself holding a private credential.
8. BUCK_K ships in two stages. **Phase 0** uses `BuckKControllerStatic` with a governance-settable value, so the identity and notes layers can land first. The on-chain PID + commodity-oracle implementation (`BuckKController`, described in the BUCK_K section) is the eventual replacement.

2 ERC Standard Analysis for BUCK_CREDIT

Before diving into the ERC-721 implementation, we compare candidate standards against the BUCK_CREDIT requirements:

Requirement	ERC-721	ERC-1155	ERC-3643 (T-REX)	ERC-721
Unique per-asset credit	Native	Possible (NFT mode)	Native	Poor
Insurer-mutable parameters	Custom role	Custom role	Built-in agent roles	Not c
Deterministic on-chain valuation	Custom view	Custom view	Custom view	share
Piecemeal client activation	Custom state	Fungible sub-amounts	Custom state	depos
Transfer restrictions	Custom	Custom	Built-in compliance	Open
Enumerable (for mint aggregation)	ERC721Enumerable	balanceOf per ID	Inherits ERC-20	Singl
Regulatory identity / KYC	Not built-in	Not built-in	ONCHAINID native	Not b
Tooling / ecosystem maturity	Excellent	Very good	Moderate (niche)	Exce

2.1 Recommendation

ERC-721 is the strongest first choice:

- Each BUCK_CREDIT is genuinely unique (specific asset, depreciation curve, insured value, premium schedule). This is precisely what ERC-721 models.
- The `ERC721Enumerable` extension provides `tokenOfOwnerByIndex()`, required for `BUCK.mint()` to aggregate all credits in an account.
- Custom extensions for dual-role access (owner activates, insurer updates) and deterministic valuation are straightforward.
- Widest tooling support, most audited implementations, lowest learning curve.

ERC-3643 is the strongest future candidate, especially if regulatory compliance (KYC, transfer restrictions, identity registry) becomes a deployment requirement. Its agent-role system maps naturally to the insurer/client relationship. However, its dependency chain (ONCHAINID, compliance modules, claim topics) adds significant complexity that is better deferred.

ERC-1155 is attractive if standardized credit "classes" emerge (e.g., all Calgary residential properties with the same depreciation model), allowing semi-fungible batching. For the initial system where each credit is individually appraised, the fungibility benefit doesn't apply.

ERC-4626 is a poor fit for individual credits but may be useful for the **InsurancePool** (where mutual insurance shares *are* fungible vault interests).

3 BUCK_CREDIT: ERC-721 Insured Asset NFT

3.1 Data Model

Each **BUCK_CREDIT** token stores the insurer's current offer of insurance and the client's activation state. All monetary values are denominated in BUCK-equivalent units (18 decimals).

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/extensions/ERC721Enumerable.sol";

contract BuckCredit is ERC721Enumerable {

    enum DepreciationType {
        NONE,           // Non-depreciating (land, gold, crypto)
        LINEAR,         // Constant annual reduction
        DECLINING_BALANCE // Percentage of remaining value per year
    }

    struct CreditParams {
        // Immutable (set at creation)
        address insurer;           // Vendor who can update this credit
        uint8  assetClass;         // Asset classification (immutable)
        uint48 createdAt;          // Creation timestamp

        // Insurer-mutable (reappraisal, schedule changes)
        uint256 faceValue;         // Maximum insured value (18 decimals)
        uint256 depreciationFloor; // Minimum value after depreciation

        DepreciationType depType;  // Depreciation model
        uint32  depRate;           // Annual rate in basis points (10000 = 100%)
        uint48  depStartAt;        // When depreciation begins

        uint32  premiumRate;       // Annual premium: basis points of activated value
        uint48  lastUpdated;        // Timestamp of last insurer update

        // Client-mutable (activation)
        uint256 activatedValue;     // Currently activated portion (<= faceValue)
        uint48  lastActivatedAt;    // Timestamp of last activation
    }

    mapping(uint256 => CreditParams) public credits;
    uint256 private _nextTokenId;

    // --- Events ---
    event CreditCreated(uint256 indexed tokenId, address indexed insurer,
        address indexed owner, uint256 faceValue);
    event CreditUpdated(uint256 indexed tokenId, address indexed insurer,
        uint256 newFaceValue, uint32 newDepRate, uint32 newPremiumRate);
    event CreditActivated(uint256 indexed tokenId, address indexed owner,
        uint256 additionalValue, uint256 totalActivated);
}
```

Key design decisions:

- **insurer** is stored per-token (not a global role), allowing multiple insurers in the same contract.

- `assetClass` is immutable – the insurer can reappraise a house’s value but cannot reclassify it.
- `activatedValue` tracks the client’s cumulative execution, capped at `faceValue`.

3.2 Depreciation Models

Depreciation is computed deterministically from the NFT’s on-chain parameters. The insurer pays gas to update parameters; the `currentValue()` view function costs no gas to query.

```

/// @notice Current depreciated value of the activated portion of this credit.
/// @dev Pure computation from on-chain state -- no oracle needed.
function currentValue(uint256 tokenId) public view returns (uint256) {
    CreditParams storage c = credits[tokenId];
    if (c.activatedValue == 0) return 0;

    uint256 depreciatedFace = _depreciate(
        c.faceValue, c.depType, c.depRate,
        c.depreciationFloor, c.depStartAt
    );

    // Activated portion depreciates proportionally
    return depreciatedFace * c.activatedValue / c.faceValue;
}

function _depreciate(
    uint256 faceValue,
    DepreciationType depType,
    uint32 depRate,          // basis points per year
    uint256 floor,
    uint48 startAt
) internal view returns (uint256) {
    if (depType == DepreciationType.NONE || block.timestamp <= startAt) {
        return faceValue;
    }

    uint256 elapsed = block.timestamp - startAt;
    uint256 depreciable = faceValue - floor; // portion subject to depreciation

    if (depType == DepreciationType.LINEAR) {
        // loss = depreciable * rate * elapsed / (365.25 days * 10000)
        uint256 loss = depreciable * depRate * elapsed / (365.25 days * 10000);
        if (loss >= depreciable) return floor;
        return faceValue - loss;
    }

    if (depType == DepreciationType.DECLINING_BALANCE) {
        // value = floor + depreciable * (1 - rate/10000) ^ years
        // Use continuous approximation: exp(-rate * elapsed / (10000 * 365.25 days))
        // Implemented via fixed-point exp approximation (see _expNeg below)
        uint256 exponent = uint256(depRate) * elapsed / (365.25 days);
        // exponent is in basis-point-years; convert to 18-decimal fixed point
        // exponent_fp = exponent * 1e18 / 10000 = exponent * 1e14
        uint256 expFp = exponent * 1e14;
        uint256 factor = _expNeg(expFp); // returns 1e18 * exp(-expFp/1e18)
        return floor + depreciable * factor / 1e18;
    }

    return faceValue; // fallback
}

/// @dev Fixed-point exp(-x) for x in 18-decimal format.
///      Uses 6th-order Taylor series, accurate to <0.01% for x < 3.0 (ie. <300% total depreciation).
///      For production, use PRBMath.exp() or ABDKMath64x64.
function _expNeg(uint256 x) internal pure returns (uint256) {
    // exp(-x) ~ 1 - x + x^2/2 - x^3/6 + x^4/24 - x^5/120 + x^6/720
    uint256 UNIT = 1e18;

```

```

    if (x > 10 * UNIT) return 0; // effectively fully depreciated

    uint256 x2 = x * x / UNIT;
    uint256 x3 = x2 * x / UNIT;
    uint256 x4 = x3 * x / UNIT;
    uint256 x5 = x4 * x / UNIT;
    uint256 x6 = x5 * x / UNIT;

    // Positive terms: 1 + x^2/2 + x^4/24 + x^6/720
    uint256 pos = UNIT + x2 / 2 + x4 / 24 + x6 / 720;
    // Negative terms: x + x^3/6 + x^5/120
    uint256 neg = x + x3 / 6 + x5 / 120;

    if (neg >= pos) return 0;
    return pos - neg;
}

```

3.2.1 Depreciation Examples

Asset	Type	Rate	Floor	30yr value (of \$100K)
Land	NONE	—	—	\$100,000
House (bldg)	LINEAR	200bp	20%	\$52,000
Vehicle	DECLINING_BALANCE	1500bp	5%	\$5,633
Farm equip	DECLINING_BALANCE	1000bp	10%	\$16,791
Gold in vault	NONE	—	—	\$100,000

A residential property would typically have *two* BUCK_CREDITS: one for the land (NONE, full face value indefinitely) and one for the structure (LINEAR or DECLINING_BALANCE, with a floor representing salvage/lot-clearing value).

3.3 Activation (Execution) Model

The client executes (activates) their credit piecemeal. Each activation increases the `activatedValue` at the *current* premium rate set by the insurer. The premium is paid in BUCKs to the insurance pool.

```

/// @notice Client activates additional credit, up to the current face value.
/// @param tokenId The BUCK_CREDIT NFT to activate
/// @param amount Additional value to activate (18 decimals)
/// @dev Caller must be the token owner.
/// Premium for the newly activated amount is collected by BUCK.mint(),
/// not here -- activation just increases the limit.
function activate(uint256 tokenId, uint256 amount) external {
    require(ownerOf(tokenId) == msg.sender, "Not credit owner");
    CreditParams storage c = credits[tokenId];
    require(c.activatedValue + amount <= c.faceValue,
        "Exceeds face value");

    c.activatedValue += amount;
    c.lastActivatedAt = uint48(block.timestamp);

    emit CreditActivated(tokenId, msg.sender, amount, c.activatedValue);
}

/// @notice Aggregate current value of all credits owned by an account.
/// @dev Called by BUCK.mint() to compute the credit limit.
function totalCurrentValue(address account) external view returns (uint256) {
    uint256 total = 0;
    uint256 count = balanceOf(account);
    for (uint256 i = 0; i < count; i++) {
        total += currentValue(tokenOfOwnerByIndex(account, i));
    }
}

```

```

    }
    return total;
}

```

The activation model separates the *decision to use credit* from the *act of minting BUCKs*. A client can activate \$200K of their \$500K home credit today, and months later activate \$100K more – possibly at a different premium rate if the insurer has updated the NFT in the interim.

3.4 Insurer Updates

The insurer can reappraise the asset, change depreciation parameters, and adjust premium schedules. These changes affect *future* credit limit computations, including already-activated amounts (since the depreciation function is computed from current parameters, not historical ones).

```

/// @notice Insurer updates credit parameters (reappraisal, schedule change).
/// @dev Only callable by the credit's designated insurer.
///      Cannot change assetClass or insurer address.
function updateCredit(
    uint256 tokenId,
    uint256 newFaceValue,
    uint256 newDepreciationFloor,
    DepreciationType newDepType,
    uint32 newDepRate,
    uint48 newDepStartAt,
    uint32 newPremiumRate
) external {
    CreditParams storage c = credits[tokenId];
    require(msg.sender == c.insurer, "Not insurer");

    // If face value decreases below activated, cap activated
    if (newFaceValue < c.activatedValue) {
        c.activatedValue = newFaceValue;
    }

    c.faceValue = newFaceValue;
    c.depreciationFloor = newDepreciationFloor;
    c.depType = newDepType;
    c.depRate = newDepRate;
    c.depStartAt = newDepStartAt;
    c.premiumRate = newPremiumRate;
    c.lastUpdated = uint48(block.timestamp);

    emit CreditUpdated(tokenId, msg.sender,
        newFaceValue, newDepRate, newPremiumRate);
}

```

Note that if the insurer reduces `faceValue` below the current `activatedValue`, the activated amount is capped downward. This could push the client's outstanding BUCK balance above their credit limit, triggering the default insurance mechanism described below.

4 IdentityRegistry: On-Chain Trust Anchor

The `IdentityRegistry` is the binding between Ethereum addresses and the privacy-preserving identity machinery described in the companion documents *Identity with Anonymity*, *Identity Example – Data Flow*, and *Cryptographic Proofs*. Every BUCK `mint`, `approve` and `transfer` consults this registry; an unregistered address cannot hold or move BUCKs.

The registry stores three pieces of state per address:

Field	Type	Source
pk[a]	G ₁ point	Identity public key generated by the wallet
E_addr[a]	(R, C) in G ₁ ²	ElGamal encryption of $M = m \cdot G$ under pk
isVerified	bool	True after PS + NIZK proofs verify on-chain
isPublic	bool	Opt-in: account publishes M (DeFi pools, etc.)

It also holds a small set of trusted issuer public keys `trustedIssuers[issuerAddr] = (X, Y)` in G_2^2 , populated by governance. These are the only PS public keys whose signatures the registry will accept at `register()` time.

4.1 Cryptographic Surface

The registry implements three on-chain verifications, all reducible to BN254 precompile calls:

1. **PS signature verification at registration:** $e(\text{sigma}'_1, X + m \cdot Y) = e(\text{sigma}'_2, g_2)$, where the verifier reconstructs the relation linearly in m via the binding NIZK in step 2. Costs $\sim 147K$ gas (3-4 `ecPairing` pairs).
2. **NIZK proof binding the rerandomized PS signature to the ElGamal ciphertext.** A Schnorr-family sigma protocol with three response scalars and a Fiat-Shamir challenge bound to $(\text{sigma}', E_{\text{new}}, \text{pk}_{\text{new}}, \text{registrant_address})$. Costs $\sim 30K$ gas of `ecMul`/`ecAdd` on top of the pairing check. Total registration cost is $\sim 235K$ gas (one-time).
3. **Chaum-Pedersen proof of re-encryption correctness at approve time.** The recipient identity ciphertext E_{bob} and proof $\text{pi}_{\text{CP}} = (e, s_1, s_2)$ arrive in `approve()` calldata; the registry reads E_{alice} from storage (anchoring the proof to the registered credential) and verifies via 3-4 `ecMul` + 3-4 `ecAdd` operations and a Fiat-Shamir reconstruction. Costs $\sim 29K$ gas per counterparty pair.

4.2 Solidity Surface

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import {BN254} from "./BN254.sol"; // helper library wrapping ecAdd/ecMul/ecPairing

contract IdentityRegistry {

    struct G1Point { uint256 X; uint256 Y; } // affine BN254 G_1
    struct G2Point { uint256[2] X; uint256[2] Y; } // affine BN254 G_2

    struct ElGamalCT { G1Point R; G1Point C; } // (R, C) = (r*G, r*pk + M)
    struct PSSig { G1Point s1; G1Point s2; } // rerandomized PS signature
    struct PSPubKey { G2Point X; G2Point Y; } // issuer (X, Y) in G_2

    // NIZK binding the PS signature to the ElGamal ciphertext (registration).
    // Six-element Schnorr-family sigma protocol; A_ps is the PS-side commitment
    // m_tilde * sigma'_1, indispensable for the pairing-product check (a).
    struct RegistrationProof {
        uint256 e; // Fiat-Shamir challenge
        uint256 s_m; // response for the identity scalar m
        uint256 s_r; // response for the ElGamal randomness r
        G1Point A_ps; // PS-side commitment: m_tilde * sigma'_1
        G1Point T_C; // ElGamal C commitment: m_tilde*G + r_tilde*pk
        G1Point T_R; // ElGamal R commitment: r_tilde * G
    }

    // Chaum-Pedersen proof of equality of plaintexts (per-approve). T1, T2, T3
```

```

// commitments must be in calldata: only T3 (Check 1) and T1-T2 (Check 2)
// are recoverable from the verification equations, so a transcript that
// binds T1 and T2 separately requires both.
struct CPPProof {
    uint256 e;           // Fiat-Shamir challenge
    uint256 s1;          // response for sk_alice
    uint256 s2;          // response for r' (re-encryption randomness for bob)
    G1Point T1;          // k1 * R_alice
    G1Point T2;          // k2 * pk_bob
    G1Point T3;          // k2 * G
}

// --- Per-address identity state ---
mapping(address => G1Point) public pk;
mapping(address => ElGamalCT) public E_addr;
mapping(address => bool) public isVerified;
mapping(address => bool) public isPublic;

// --- Trusted issuers (set by governance) ---
mapping(address => PPSKey) public trustedIssuers;
address public governance;

event Registered(address indexed account, address indexed issuer);
event PublicSet(address indexed account, bool isPublic);
event IssuerTrusted(address indexed issuer);

/// @notice One-time per-address registration with PS+NIZK proofs (~235K gas).
/// @param issuer Address of the issuer whose PS signature appears in sigma'.
/// @param pk Identity public key (G_1) generated by the wallet.
/// @param E ElGamal ciphertext (R, C) = (r*G, r*pk + M) of identity point M.
/// @param sigma Rerandomized PS signature (sigma'_1, sigma'_2) on m = H(identity_data).
/// @param proof NIZK proof binding sigma to E under pk.
function register(
    address issuer,
    G1Point calldata pk,
    ElGamalCT calldata E,
    PSSig calldata sigma,
    RegistrationProof calldata proof
) external {
    require(!isVerified[msg.sender], "already registered");
    require(isTrustedIssuer[issuer], "untrusted issuer");

    // (1) Recompute Fiat-Shamir challenge with full context binding.
    uint256 e_check = BN254.fsRegistration(
        sigma, E, pk, proof.A_ps, proof.T_C, proof.T_R, msg.sender
    );
    require(e_check == proof.e, "bad NIZK challenge");

    // (2) Verify NIZK consistency in G_1 (ecMul/ecAdd, ~30K gas).
    require(
        BN254.checkRegistrationNIZK(pk, E, sigma, proof, _trustedIssuers[issuer]),
        "bad NIZK proof"
    );

    // (3) Verify PS signature relation via ecPairing precompile (~147K gas).
    require(
        BN254.checkPSSignature(sigma, _trustedIssuers[issuer], proof),
        "bad PS signature"
    );

    _pk[msg.sender] = pk;
    _E_addr[msg.sender] = E;
    isVerified[msg.sender] = true;
    issuerOf[msg.sender] = issuer;
    emit Registered(msg.sender, issuer);
}

/// @notice Opt in/out of "public" identity (M is published, no per-tx CP required).

```

```

///      Used for AMM pools, treasury accounts, and other DeFi infrastructure
///      that control their own private key.
function setPublic(bool _isPublic) external {
    require(isVerified[msg.sender], "not registered");
    isPublic[msg.sender] = _isPublic;
    emit PublicSet(msg.sender, _isPublic);
}

/// @notice Governance marks a system contract (e.g. Notes, InsurancePool) as public
///      without requiring it to self-register. System contracts have no private
///      key to generate CP proofs with, but they are well-known trusted counterparties.
function setSystemPublic(address account, bool _isPublic) external {
    require(msg.sender == governance, "not governance");
    require(account != address(0), "account=0");
    isPublic[account] = _isPublic;
    emit PublicSet(account, _isPublic);
}

/// @notice Verify a Chaum-Pedersen proof that E_bob re-encrypts the identity
///      registered to 'sender' under 'spender's public key.
/// @dev   Called by Buck.approve(); E_alice is read from storage here.
///      ~29K gas of ecMul/ecAdd + Fiat-Shamir reconstruction.
function verifyApprove(
    address sender,
    address spender,
    ElGamalCT calldata E_bob,
    CPProof calldata pi
) external view returns (bool) {
    require(isVerified[sender], "sender not verified");
    require(isVerified[spender], "spender not verified");
    ElGamalCT storage E_alice = _E_addr[sender];
    return BN254.checkChaumPedersen(
        E_alice, E_bob, _pk[sender], _pk[spender], pi, sender, spender
    );
}

/// @notice Governance whitelists an authorized issuer's PS public key.
function trustIssuer(address issuer, PSPubKey calldata ipk) external {
    require(msg.sender == governance, "not governance");
    _trustedIssuers[issuer] = ipk;
    isTrustedIssuer[issuer] = true;
    emit IssuerTrusted(issuer);
}

/// @notice Governance revokes a previously trusted issuer. Does not un-register
///      existing accounts; only prevents new registrations under that issuer.
function revokeIssuer(address issuer) external {
    require(msg.sender == governance, "not governance");
    require(isTrustedIssuer[issuer], "not trusted");
    isTrustedIssuer[issuer] = false;
    delete _trustedIssuers[issuer];
    emit IssuerRevoked(issuer);
}

function transferGovernance(address next) external {
    require(msg.sender == governance, "not governance");
    require(next != address(0), "next=0");
    address prev = governance;
    governance = next;
    emit GovernanceTransferred(prev, next);
}
}

```

4.3 Public-Claim Accounts

DeFi infrastructure – AMM pools, vaults, oracles – cannot generate Chaum-Pedersen proofs at `approve` time, because the contract has no private key. These accounts call `setPublic(true)` after `register()`; from then on, counterparties of a public account skip the proof requirement and the on-chain receipt is constructed directly from the published identity point. Public accounts forfeit content privacy: their identity M is observable on-chain, but they retain unforgeability and the audit trail.

4.4 Issuer Trust

`trustedIssuers` is the only governance-controlled trust anchor in the system. Adding a new issuer admits a new origin of registrable identities; removing one stops new registrations under that issuer (existing registrations are unaffected – they were verified at registration time and are never re-verified). Epoch-based credential renewal, key revocation, and issuer rotation are described in the Identity: Epoch-Based Credential Renewal document.

5 BUCK: ERC-20 Token

The BUCK token is an **identity-aware** ERC-20. Beyond the usual `mint` / `transfer` / `approve` surface, every state-changing call routes through the `IdentityRegistry`:

- `mint()` refuses to issue BUCKs to an unverified address.
- `approve()` takes the recipient’s re-encrypted identity ciphertext and a Chaum-Pedersen NIZK proof, and stores a *receipt fragment* binding the (sender, spender) pair to that ciphertext.
- `transfer()` and `transferFrom()` enforce a **bilateral** identity check using the receipt fragments and the public-claim flag, then emit a `BuckTransferReceipt` carrying only ciphertext hashes – no plaintext identity ever touches the chain.

This is the on-chain enforcement of the mutual-decryptability invariant: every transfer leaves behind a permanent record from which both counterparties (and only those counterparties, by decrypting their respective ciphertext) can reconstruct the identity of the other.

5.1 `mint()` Implementation

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

interface IBuckK {
    function currentBuckK() external view returns (uint256);
}

interface IBuckCredit {
    function totalCurrentValue(address holder) external view returns (uint256);
}

interface IIdentityRegistry {
    struct G1Point { uint256 X; uint256 Y; }
    struct ElGamalCT { G1Point R; G1Point C; }
    struct CPPProof {
        uint256 e; uint256 s1; uint256 s2;
    }
}
```

```

    G1Point T1; G1Point T2; G1Point T3;
}

function isVerified(address) external view returns (bool);
function isPublic(address) external view returns (bool);
function verifyApprove(
    address sender, address spender,
    ElGamalCT calldata E_bob, CPPProof calldata pi
) external view returns (bool);
}

contract Buck is ERC20 {

    IBuckCredit public immutable buckCredit;
    IBuckK public immutable buckK;
    IIdentityRegistry public immutable identity;
    address public immutable insurancePool;

    uint256 constant PRECISION = 1e18;

    // Per-account stored credit limit (monotonically increases per mint)
    mapping(address => uint256) public storedLimit;

    // Receipt fragments: keccak256(E_recipient_ciphertext) per (sender, spender) pair.
    // Set by approve(); consumed by transferFrom() and the mirrored transfer() path.
    mapping(address => mapping(address => bytes32)) internal _receiptFragments;

    event Minted(address indexed account, uint256 amount, uint256 premium,
        uint256 creditValue, uint256 buckKValue, uint256 newLimit);

    /// @dev Receipt event: ciphertext *hashes* only. Counterparties verify off-chain
    /// by recomputing keccak256 over their stored E_alice / E_bob.
    event BuckTransferReceipt(
        address indexed from,
        address indexed to,
        uint256 amount,
        bytes32 fromCipherHash,
        bytes32 toCipherHash
    );

    constructor(
        address _buckCredit,
        address _buckK,
        address _identity,
        address _insurancePool
    ) ERC20("Alberta Buck", "BUCK") {
        buckCredit = IBuckCredit(_buckCredit);
        buckK = IBuckK(_buckK);
        identity = IIdentityRegistry(_identity);
        insurancePool = _insurancePool;
    }

    /// @notice Mint BUCKs against the caller's aggregated BUCK_CREDIT value.
    /// @dev Caller must hold a verified IdentityRegistry credential.
    /// @param amount Desired BUCK amount to mint (18 decimals).
    function mint(uint256 amount) external {
        require(identity.isVerified(msg.sender), "BUCK: sender not verified");

        // 1. Aggregate all BUCK_CREDITS in the caller's account
        uint256 totalCreditValue = buckCredit.totalCurrentValue(msg.sender);

        // 2. Get current BUCK_K stabilization factor (Phase 0: static getter)
        uint256 currentBuckK = buckK.currentBuckK();

        // 3. Compute maximum credit limit
        uint256 maxLimit = totalCreditValue * currentBuckK / PRECISION;

        // 4. Only increase the stored limit (never decrease via mint)

```

```

    if (maxLimit > storedLimit[msg.sender]) {
        storedLimit[msg.sender] = maxLimit;
    }

    // 5. Check that mint doesn't exceed limit
    uint256 currentBalance = balanceOf(msg.sender);
    require(currentBalance + amount <= storedLimit[msg.sender],
        "Exceeds credit limit");

    // 6. Compute default insurance premium
    uint256 premium = _computePremium(
        msg.sender, amount, storedLimit[msg.sender]
    );

    // 7. Mint: net amount to client, premium to insurance pool
    _mint(msg.sender, amount - premium);
    if (premium > 0) {
        _mint(insurancePool, premium);
    }

    emit Minted(msg.sender, amount, premium,
        totalCreditValue, currentBuckK, storedLimit[msg.sender]);
}

/// @notice Burn BUCKs to reduce outstanding balance (repayment).
function burn(uint256 amount) external {
    _burn(msg.sender, amount);
}

```

5.2 Identity-Aware approve()

The standard ERC-20 approve(spender, amount) becomes:

```

/// @notice Approve 'spender' to transfer up to 'amount' BUCKs from the caller's balance,
///         and simultaneously record a Chaum-Pedersen-bound identity receipt fragment.
/// @dev    The proof 'pi_CP' proves that 'E_bob' re-encrypts the *same identity point M*
///         that the caller registered in IdentityRegistry, under spender's public key.
///         IdentityRegistry reads E_alice from its own storage, anchoring the proof.
/// @param  spender The address being approved as a transfer agent.
/// @param  amount   ERC-20 allowance.
/// @param  E_bob    ElGamal ciphertext of the caller's identity, encrypted under pk_spender.
/// @param  pi_CP    Chaum-Pedersen NIZK proof of equal-plaintext re-encryption.
function approve(
    address spender,
    uint256 amount,
    IIdentityRegistry.ElGamalCT calldata E_bob,
    IIdentityRegistry.CPPProof calldata pi_CP
) public returns (bool) {
    require(identity.isVerified(msg.sender), "BUCK: sender not verified");
    require(
        identity.isVerified(spender) || identity.isPublic(spender),
        "BUCK: spender not verified"
    );

    // Public spenders (AMM pools, vaults) skip CP -- their identity is already on-chain.
    if (!identity.isPublic(spender)) {
        require(
            identity.verifyApprove(msg.sender, spender, E_bob, pi_CP),
            "BUCK: bad CP proof"
        );
        _receiptFragments[msg.sender][spender] = keccak256(
            abi.encode(E_bob.R.X, E_bob.R.Y, E_bob.C.X, E_bob.C.Y)
        );
    } else {
        // Public counterparty: receipt is a hash of the well-known M instead.
        _receiptFragments[msg.sender][spender] = _publicReceiptHash(spender);
    }
}

```

```

    _approve(msg.sender, spender, amount);
    return true;
}

/// @dev Block the parameterless ERC-20 approve; identity-bound approve is mandatory.
function approve(address, uint256) public pure override returns (bool) {
    revert("BUCK: use identity-bound approve");
}

```

The approve call costs roughly 29K (CP verify) + 46K (standard ERC-20 approve) + 22K (receipt SSTORE) = ~97K gas, paid once per (sender, spender) pair. Subsequent transfers between the same counterparties incur only the bilateral check and the receipt event (~5K gas of additional work above a baseline ERC-20 transfer).

5.3 Identity-Aware transfer() and transferFrom()

```

/// @notice Transfer BUCKs to 'to'.
/// @dev Requires bilateral identity verification:
/// - Both addresses are isVerified, and
/// - Either (a) 'to' is isPublic, or
/// (b) caller has a stored receipt fragment for 'to' from a prior approve().
function transfer(address to, uint256 amount) public override returns (bool) {
    _identityCheckedTransfer(msg.sender, to, amount);
    return true;
}

/// @notice TransferFrom path used by spenders (AMM router pulling Alice's BUCKs).
function transferFrom(address from, address to, uint256 amount)
    public override returns (bool)
{
    _spendAllowance(from, msg.sender, amount);
    _identityCheckedTransfer(from, to, amount);
    return true;
}

function _identityCheckedTransfer(address from, address to, uint256 amount) internal {
    require(identity.isVerified(from), "BUCK: sender not verified");
    require(
        identity.isVerified(to) || identity.isPublic(to),
        "BUCK: recipient not verified"
    );

    bytes32 toHash;
    if (identity.isPublic(to)) {
        // Public recipient: receipt anchored to recipient's published M.
        toHash = _publicReceiptHash(to);
    } else {
        // Private recipient: caller must have a Chaum-Pedersen-bound receipt fragment.
        toHash = _receiptFragments[from][to];
        require(toHash != bytes32(0), "BUCK: missing identity receipt");
    }

    bytes32 fromHash = identity.isPublic(from)
        ? _publicReceiptHash(from)
        : _receiptFragments[to][from]; // mirrored fragment if 'to' previously approved 'from'
    // For a one-shot transfer with no prior reciprocal approve, fromHash may be zero.
    // The receipt event still records the asymmetric attestation; auditors reading the
    // event know which side was Chaum-Pedersen bound.

    _transfer(from, to, amount);
    emit BuckTransferReceipt(from, to, amount, fromHash, toHash);
}

function _publicReceiptHash(address account) internal view returns (bytes32) {

```

```

(IIdentityRegistry.G1Point memory R, IIdentityRegistry.G1Point memory C)
    = identity.E_addr(account);
return keccak256(abi.encode(R.X, R.Y, C.X, C.Y, "PUBLIC"));
}

```

The `BuckTransferReceipt` event is the auditable trail: `fromCipherHash` and `toCipherHash` are keccak256 commitments over the corresponding ElGamal ciphertexts. Counterparties can match a receipt to their off-chain copies of E_{alice}, E_{bob} by recomputing the hash; observers without the underlying ciphertexts learn nothing beyond the fact of a transfer.

5.4 Default Insurance Premium

The premium scales with utilization: the closer the account’s outstanding BUCKs approach the credit limit, the more expensive each additional BUCK becomes. This mirrors the architecture document’s description of Parametric Default Insurance.

```

/// @dev Premium as a fraction of the minted amount, scaling with utilization.
///      Uses a quadratic curve: premium_rate = baseRate + (utilization^2) * scaleRate
///      At 0% utilization: ~0.5% premium
///      At 50% utilization: ~1.75% premium
///      At 90% utilization: ~4.55% premium
///      At 100% utilization: impossible (can't mint past limit)
uint256 constant BASE_RATE = 50;    // 0.50% base (in basis points)
uint256 constant SCALE_RATE = 450;  // 4.50% additional at 100% utilization
uint256 constant BP = 10000;

function _computePremium(
    address account,
    uint256 mintAmount,
    uint256 limit
) internal view returns (uint256) {
    if (limit == 0) return 0;

    // Utilization after this mint
    uint256 newBalance = balanceOf(account) + mintAmount;
    uint256 utilization = newBalance * PRECISION / limit; // 0 to 1e18

    // premium_rate = BASE_RATE + utilization^2 * SCALE_RATE / 1e18
    uint256 utilSq = utilization * utilization / PRECISION;
    uint256 rate = BASE_RATE + utilSq * SCALE_RATE / PRECISION;

    return mintAmount * rate / BP;
}

```

The quadratic scaling creates a natural disincentive against over-leveraging. A client using 50% of their credit pays roughly 1.75% of each minted BUCK to the insurance pool; at 90% they pay 4.55%. The insurance pool accumulates these premiums to cover defaults (accounts where asset depreciation or `BUCK_K` changes push outstanding BUCKs above the credit limit).

6 BUCK_K: Value Stabilization Controller

The `BUCK_K` contract maintains purchasing-power parity between the BUCK and its commodity basket. It ships in two stages behind a shared one-function interface (`currentBuckK()` `external view returns (uint256)`) so the Buck contract is invariant to which stage is deployed:

- **Phase 0** – `BuckKControllerStatic`: a governance-settable constant. Constructor (`uint256 _buckK, address _governance`); functions `currentBuckK()`, `compute()` (no-op, returns the

stored value), `setBuckK(uint256)` (governance-only), `transferGovernance(address)`. This is the version currently deployed alongside the identity and notes layers so those layers can land first.

- **Phase Later – BuckKController:** a full PID implementation, identical in structure to the project’s ownercredit PID controller, ported to Solidity fixed-point arithmetic. It reads a commodity basket from Chainlink feeds and the BUCK/USDC price from a Uniswap V3 TWAP, runs a P/I/D update on `compute()` once per `dT` seconds, and writes the new `buckK`.

The PID implementation is documented in full below; the Buck ERC-20 reads only `currentBuckK()` from either variant, so swapping one for the other is a constructor argument.

6.1 Process Variable and Setpoint

PID Term	Source	Meaning
Process Variable	BUCK/USDC Uniswap V3 TWAP	What a BUCK actually trades for
Setpoint	Commodity basket oracle sum	What a BUCK <i>should</i> be worth (basket cost)
Error	Setpoint - Process	Positive = BUCK undervalued, negative = overvalued
Output (BUCK_K)	PID computation	Credit limit multiplier (1.0 = neutral)

When BUCK trades *below* basket value (positive error), BUCK_K increases, expanding everyone’s credit limit. This floods the market with cheap BUCKs, which arbitrageurs buy to acquire underpriced commodities, driving the BUCK price back up. The converse reduces BUCK_K when BUCKs are overvalued.

6.2 Solidity Implementation

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import "@chainlink/contracts/src/v0.8/interfaces/AggregatorV3Interface.sol";

contract BuckKController {

    // --- PID Gains (governance-set, 18-decimal fixed point) ---
    int256 public Kp;
    int256 public Ki;
    int256 public Kd;

    // --- PID State (updated on compute() when dT has elapsed) ---
    int256 public P;          // Proportional error
    int256 public I;          // Integral accumulator
    int256 public D;          // Derivative
    uint256 public lastUpdate; // Timestamp of last PID state update

    // --- Output ---
    uint256 public buckK;      // Current BUCK_K output (18-decimal, 1e18 = 1.0)
    uint256 public dT;         // Minimum seconds between PID state updates

    // --- Output Limits (anti-windup) ---
    uint256 public buckKMin;   // Floor (e.g., 0.5e18 = 50% of neutral)
    uint256 public buckKMax;   // Ceiling (e.g., 2.0e18 = 200% of neutral)

    // --- Oracle Configuration ---
    address public buckUsdcPool; // Uniswap V3 BUCK/USDC pool
    uint32 public twapInterval;  // TWAP window in seconds (e.g., 1800 = 30min)

    // Commodity basket: Chainlink price feeds + weights
```

```

struct BasketComponent {
    AggregatorV3Interface feed; // Chainlink price feed
    uint256 weight;             // Weight in basket (18 decimals, sum = 1e18)
    uint8 feedDecimals;         // Decimals of the Chainlink feed
}
BasketComponent[] public basket;

int256 constant UNIT = 1e18;
address public governance;

event BuckKUpdated(uint256 newBuckK, int256 error, int256 P, int256 I, int256 D);
event GainsUpdated(int256 Kp, int256 Ki, int256 Kd);

constructor(
    int256 _Kp, int256 _Ki, int256 _Kd,
    uint256 _dT,
    uint256 _buckKMin, uint256 _buckKMax,
    address _buckUsdcPool, uint32 _twapInterval,
    address _governance
) {
    Kp = _Kp; Ki = _Ki; Kd = _Kd;
    dT = _dT;
    buckKMin = _buckKMin;
    buckKMax = _buckKMax;
    buckUsdcPool = _buckUsdcPool;
    twapInterval = _twapInterval;
    governance = _governance;
    buckK = uint256(UNIT); // Start at 1.0 (neutral)
    lastUpdate = block.timestamp;
}

/// @notice Compute and return the current BUCK_K value.
/// @dev If dT has elapsed since last update, performs a full PID cycle
///      (reads oracles, updates P/I/D state, stores new buckK).
///      Otherwise returns the cached buckK value.
///      Called by BUCK.mint() -- the minter pays gas for any PID update.
function compute() external returns (uint256) {
    if (block.timestamp - lastUpdate < dT) {
        return buckK;
    }

    // --- Read oracles ---
    int256 basketCost = _getBasketCost(); // Setpoint: USDC cost of basket
    int256 buckPrice = _getBuckPrice();    // PV: BUCK market price in USDC

    // --- PID computation ---
    int256 dt = int256(block.timestamp - lastUpdate);
    int256 error = basketCost - buckPrice; // Positive = BUCK undervalued

    int256 newP = error;
    int256 newI = I + error * dt / UNIT; // Scaled integral
    int256 newD = dt > 0
        ? (error - P) * UNIT / dt // Rate of change
        : int256(0);

    int256 rawOutput = UNIT // Start at 1.0 (neutral)
        + newP * Kp / UNIT
        + newI * Ki / UNIT
        + newD * Kd / UNIT;

    // --- Anti-windup clamping ---
    uint256 newBuckK;
    if (rawOutput < int256(buckKMin)) {
        newBuckK = buckKMin;
        // Only update integral if it would push output away from saturation
        if (newI > I) I = newI;
    } else if (rawOutput > int256(buckKMax)) {
        newBuckK = buckKMax;
    }
}

```

```

        if (newI < I) I = newI;
    } else {
        newBuckK = uint256(rawOutput);
        I = newI;
    }

    P = newP;
    D = newD;
    buckK = newBuckK;
    lastUpdate = block.timestamp;

    emit BuckKUpdated(newBuckK, error, P, I, D);
    return newBuckK;
}

/// @notice Current BUCK_K without updating state (view-only).
function currentBuckK() external view returns (uint256) {
    return buckK;
}

// --- Oracle Helpers ---

/// @dev Sum of weighted commodity prices from Chainlink feeds, in USDC (18 decimals).
function _getBasketCost() internal view returns (int256) {
    int256 total = 0;
    for (uint i = 0; i < basket.length; i++) {
        BasketComponent storage comp = basket[i];
        (, int256 price,,) = comp.feed.latestRoundData();
        // Normalize to 18 decimals
        int256 normalized = price * int256(10 ** (18 - comp.feedDecimals));
        total += normalized * int256(comp.weight) / UNIT;
    }
    return total;
}

/// @dev BUCK/USDC price from Uniswap V3 TWAP oracle (18 decimals).
/// Uses the pool's observe() function for manipulation resistance.
function _getBuckPrice() internal view returns (int256) {
    // Uniswap V3 TWAP implementation:
    // 1. Call pool.observe([twapInterval, 0])
    // 2. Compute tick = (tickCumulatives[1] - tickCumulatives[0]) / twapInterval
    // 3. Convert tick to price: price = 1.0001^tick * 10^(token1Decimals - token0Decimals)
    //
    // Omitted here for brevity -- production would use Uniswap's OracleLibrary.
    // See: https://docs.uniswap.org/contracts/v3/guides/providing-liquidity/get-a-price
    revert("_getBuckPrice: implement with OracleLibrary");
}

// --- Governance ---

function setGains(int256 _Kp, int256 _Ki, int256 _Kd) external {
    require(msg.sender == governance, "Not governance");
    Kp = _Kp; Ki = _Ki; Kd = _Kd;
    emit GainsUpdated(_Kp, _Ki, _Kd);
}

function setDT(uint256 _dT) external {
    require(msg.sender == governance, "Not governance");
    dT = _dT;
}

function addBasketComponent(address feed, uint256 weight, uint8 decimals) external {
    require(msg.sender == governance, "Not governance");
    basket.push(BasketComponent({
        feed: AggregatorV3Interface(feed),
        weight: weight,
        feedDecimals: decimals
    }));
}

```

```

    }
}

```

6.3 PID Tuning Considerations

The PID gains determine how aggressively BUCK_K responds to price deviations:

Gain	Effect of Increasing	Risk of Over-tuning
Kp	Faster response to price deviations	Oscillation, over-correction
Ki	Eliminates steady-state error (persistent mispricing)	Integral windup, slow recovery
Kd	Dampens oscillation, anticipates trends	Noise amplification

Initial gains should be conservative (high Kd relative to Kp, low Ki) to favor stability over responsiveness. The `dT` parameter (minimum update interval) acts as an additional dampening mechanism – a 1-hour `dT` means BUCK_K changes at most once per hour regardless of how many mints occur, preventing high-frequency oscillation.

The TWAP interval (`twapInterval`) provides manipulation resistance: a 30-minute TWAP means an attacker would need to sustain a price manipulation for 30 minutes to meaningfully affect BUCK_K – a prohibitively expensive proposition on a liquid pool.

6.4 Adaptive Gain Scheduling (Future Enhancement)

The current implementation uses governance-set fixed gains. A future enhancement could implement on-chain gain scheduling where Kp, Ki, and Kd are adjusted based on observed error dynamics:

- **Low volatility regime** (error variance below threshold): reduce Kp, increase Ki for tighter long-term tracking.
- **High volatility regime** (error variance above threshold): increase Kp and Kd for faster response, reduce Ki to prevent windup.
- **Regime detection**: maintain a rolling variance of the error signal over a window of N updates.

This maps directly to the `PIDController.reset()` pattern in the JavaScript implementation, where gains can be hot-swapped without spiking the derivative term.

7 Notes: Privacy-Preserving Denomination

BUCK transfers carry a public (`from`, `to`, `amount`) quartet. For private value movement, holders denominate their BUCKs as **notes**: a pair of Poseidon-hash commitments posted to the `Notes` contract, backed by an equal amount of BUCK locked in that contract. Notes do not leak values, flavours, or identity bindings – the public state is just a list of opaque 32-byte commitments and a running face-value total.

The design is specified in full in Notes and the underlying SNARK construction in Proofs. This section documents the Ethereum surface: the `Notes` contract, the `IMintVerifier` interface, the circom Groth16 mint circuit, and the adapter that bridges the two.

7.1 Mint Circuit (circom, Groth16 on BN254)

`circuits/mint.circom` is pinned at $N = 2$ commitments per batch for the Phase 2 deployment. A variable- N deployment re-templates both the circuit and the `MintVerifierAdapter`'s public-signals arity.

Signal	Role	Description
<code>totalFace</code>	public	Sum of note values, in BUCK (18 decimals)
<code>cm[i]</code>	public	Poseidon(5)(flavor, v, rho, idHash, predicate) for each note i
<code>flavor[i]</code>	private	Note flavour (A1/A2/B1 from Notes design; 2 == A2 in tests)
<code>v[i]</code>	private	Note value (18-decimal integer, sum of <code>v[i]</code> must equal <code>totalFace</code>)
<code>rho[i]</code>	private	Per-note randomness
<code>idHash[i]</code>	private	Hash of the identity binding for this note
<code>predicate[i]</code>	private	Predicate commitment (zero for the plain mint path)

Constraints: 2 Poseidon-hash checks (`cm[i] == Poseidon(5)(...)`) and one sum check (`totalFace == sum v[i]`). `snarkjs` compiles this to 648 non-linear + 1023 linear R1CS constraints; a ceremonial 2^{12} powers-of-tau and `groth16` phase-2 setup under `scripts/snark/setup.sh` emits `src/MintGroth16Verifier.s` the auto-generated `snarkjs` verifier:

```
interface IMintGroth16 {
  function verifyProof(
    uint256[2]      calldata a,
    uint256[2][2]   calldata b,
    uint256[2]      calldata c,
    uint256[3]      calldata pubSignals    // [totalFace, cm[0], cm[1]]
  ) external view returns (bool);
}
```

The trusted-setup contributions in `setup.sh` use a fixed dev-only entropy string. A production deployment must run a real multi-party ceremony; the same script is the scaffolding.

7.2 IMintVerifier and MintVerifierAdapter

The `Notes` contract is decoupled from the specific SNARK toolchain via a narrow interface:

```
interface IMintVerifier {
  /// @return ok true iff (proof, totalFace, commitments) is a valid mint witness for 'issuer'
  function verifyMint(
    bytes      calldata proof,
    uint256    totalFace,
    uint256[]  calldata commitments,
    address    issuer
  ) external view returns (bool);
}
```

Two implementations ship today:

- `StubMintVerifier` (Phase 1): returns a governance-controlled boolean. Used in the initial deployment and all unit tests that exercise state transitions unrelated to the SNARK itself.

- `MintVerifierAdapter` (Phase 2): decodes proof as `abi.encode(uint256[2], uint256[2][2], uint256[2])`, rebuilds the `pubSignals` array as `[totalFace, commitments[0], commitments[1]]`, and forwards to `MintGroth16Verifier.verifyProof`. `MINT_BATCH_SIZE` is a public constant (2).

Swapping the verifier is a single governance call (`Notes.setMintVerifier(address)`); the `Notes` contract's own storage and event surface are invariant.

7.3 Notes Contract

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity ^0.8.20;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IMintVerifier} from "./IMintVerifier.sol";

contract Notes {
    IERC20 public immutable buck;
    IMintVerifier public mintVerifier;
    address public governance;

    // Public transparency state.
    uint256[] public commitments; // ordered list of cm values
    mapping(uint256 => bool) public commitmentExists; // duplicate-rejection index
    mapping(uint256 => bool) public nullifiers; // reserved for Phase 3 spend path
    uint256 public noteFaceSum; // sum of all v_i ever minted

    event Minted (address indexed issuer, uint256 totalFace, uint256 startIndex, uint256 count);
    event Appended (uint256 indexed cm, uint256 indexed leafIndex);
    event MintVerifierUpdated (address indexed previous, address indexed next);
    event GovernanceTransferred(address indexed previous, address indexed next);

    constructor(address _buck, address _verifier, address _governance) {
        require(_buck != address(0), "buck=0");
        require(_verifier != address(0), "verifier=0");
        require(_governance != address(0), "gov=0");
        buck = IERC20(_buck);
        mintVerifier = IMintVerifier(_verifier);
        governance = _governance;
    }

    /// @notice Escrow 'totalFace' BUCK and append 'cms' to the commitment pool.
    /// @dev The caller must first approve(this, totalFace) on BUCK. Because Notes
    /// is registered setSystemPublic(true), the transferFrom satisfies BUCK's
    /// identity-aware transfer rules without requiring Notes to hold a CP-bound
    /// receipt fragment for the caller.
    function mint(
        bytes calldata proof,
        uint256[] calldata cms,
        uint256 totalFace
    ) external {
        require(cms.length > 0, "Notes: empty batch");
        require(
            mintVerifier.verifyMint(proof, totalFace, cms, msg.sender),
            "Notes: bad mint proof"
        );

        // Pull the face value from the caller. Reverts if allowance or balance insufficient.
        require(buck.transferFrom(msg.sender, address(this), totalFace), "Notes: transfer failed");

        uint256 startIndex = commitments.length;
        for (uint256 i = 0; i < cms.length; i++) {
            uint256 cm = cms[i];
            require(cm != 0, "Notes: zero commitment");
            require(!commitmentExists[cm], "Notes: duplicate commitment");
        }
    }
}
```

```

        commitmentExists[cm] = true;
        commitments.push(cm);
        emit Appended(cm, startIndex + i);
    }
    noteFaceSum += totalFace;
    emit Minted(msg.sender, totalFace, startIndex, cms.length);
}

function commitmentCount() external view returns (uint256) { return commitments.length; }

// --- Governance ---
function setMintVerifier(address next) external { /* governance-gated */ }
function transferGovernance(address next) external { /* governance-gated */ }
}

```

Phase 2 closes the happy path (credit-backed BUCKs in -> opaque commitments out, with a real SNARK binding the sum). The **spend** path – nullifier reveals plus per-output commitments under a second SNARK – is the subject of Notes Phase 3; the `nullifiers[]` mapping above is the reserved on-chain state for that phase.

8 Integration: Full mint() and transfer() Walkthrough

Consider Alice, who owns a \$500K Calgary home (land: \$200K, structure: \$300K). Two insurers (LandCo, HomeCo) have issued BUCK_CREDITS:

Token	Insurer	Face Value	Depreciation	Activated	Current Value
#42	LandCo	200,000	NONE	200,000	200,000
#43	HomeCo	300,000	LINEAR, 200bp, 20% fl	150,000	141,000
Total					341,000

The structure has depreciated 6% over 3 years (LINEAR at 2%/yr applied to the 80% depreciable portion). Alice activated all of the land credit but only half of the structure credit.

8.1 Step 0: Identity Registration (One-Time)

Before Alice can interact with BUCK at all, she must hold a verified identity credential:

1. **KYC ceremony** (off-chain): a trusted issuer (the Alberta government, ATB Financial, etc.) verifies Alice’s documents and signs her identity scalar $m = H(identity_data)$ with their PS key, producing $\sigma = (\sigma_1, \sigma_2)$. Alice stores (m, σ) in her wallet.
2. **Derivation** (off-chain): Alice’s wallet rerandomizes σ into σ' , generates a fresh identity key pair (sk_{alice}, pk_{alice}) , encrypts her identity point $M = m \cdot G$ under pk_{alice} , and produces a NIZK proof π binding σ' to that ciphertext.
3. **On-chain registration**: Alice calls `IdentityRegistry.register(issuer, pk_alice, E_alice, sigma', pi)`. The contract verifies π and the PS pairing relation via the BN254 precompiles (~235K gas, paid once for the lifetime of this Ethereum address). After this call, `isVerified[alice] = true` and `issuerOf[alice] = issuer`.

Alice repeats steps 2-3 for every Ethereum account she controls; the issuer never sees those accounts.

8.2 Step 1: Mint

Alice calls `BUCK.mint(50000e18)`:

1. The contract first checks `identity.isVerified(alice) = true`.
2. `buckCredit.totalCurrentValue(alice)` returns `341,000e18`.
3. `buckK.currentBuckK()` returns the current stabilization factor. **Phase 0:** this is just the static value the governance multisig last wrote. **Eventual PID build:** the controller fetches oracles – e.g. commodity basket at 1.02 USDC, BUCK at 0.99 USDC, error +0.03 -> `BUCK_K = 1.015`.
4. `maxLimit = 341,000 * BUCK_K`. Assume `BUCK_K = 1.015`, so `maxLimit = 346,115`.
5. Alice’s current balance is 280,000 BUCKs. After mint: `330,000 < 346,115`. OK.
6. Utilization after mint: `330,000 / 346,115 = 95.3%`.
 - Premium rate: $0.50\% + (0.953^2 * 4.50\%) = 4.59\%$.
 - Premium: `50,000 * 4.59% = 2,295` BUCKs to insurance pool.
7. Alice receives 47,705 BUCKs. Insurance pool receives 2,295 BUCKs.

8.3 Step 2: Approve Bob (Identity Handshake)

Alice wants to pay Bob 10,000 BUCKs through the same router pattern most ERC-20 flows use (`approve + transferFrom`). Bob is also registered.

1. Alice’s wallet reads Bob’s `pk_bob` from `IdentityRegistry`.
2. Off-chain, Alice’s wallet decrypts `E_alice` (it knows `sk_alice`), recovers `M`, and re-encrypts under `pk_bob` with fresh randomness `r_b`: $E_{bob} = (r_b \cdot G, r_b \cdot pk_{bob} + M)$.
3. The wallet produces a Chaum-Pedersen NIZK proof $\pi_{CP} = (e, s_1, s_2)$ that `E_bob` encrypts the same `M` registered in `E_alice`. The Fiat-Shamir challenge binds `msg.sender = alice`, `spender = bob`, and `block.chainid`.
4. Alice calls `BUCK.approve(bob, 10000e18, E_bob, pi_CP)`. The contract:
 - checks both parties are verified;
 - asks `IdentityRegistry.verifyApprove(alice, bob, E_bob, pi_CP)` – the registry reads `E_alice` from its own storage (so Alice can’t substitute a fake credential) and runs the CP check (~29K gas);
 - stores `_receiptFragments[alice][bob] = keccak256(E_bob)`;
 - records the standard ERC-20 allowance.
5. Off-chain, Alice’s Holochain hApp delivers the matching `identity_data` and `E_bob` ciphertext to Bob’s hApp. Bob’s wallet decrypts to recover `M'` and asserts $H(identity_data) \cdot G \stackrel{?}{=} M'$ – the two-channel binding from the Identity document.

8.4 Step 3: Bob Pulls the Transfer

Bob (or a contract Bob authorized) calls `BUCK.transferFrom(alice, bob, 10000e18)`:

1. `_spendAllowance(alice, bob, 10000e18)` debits the allowance Alice set above.
2. `_identityCheckedTransfer(alice, bob, 10000e18)` verifies both parties are still verified and that `_receiptFragments[alice][bob] != 0` (Alice did supply a CP-bound receipt at approve time).
3. Tokens transfer.
4. `BuckTransferReceipt(alice, bob, 10000e18, fromHash, toHash)` is emitted, with `fromHash` and `toHash` being keccak256 commitments over the corresponding ciphertexts. An auditor with subpoena power over either Alice or Bob can recover the underlying identities; an outside observer learns only that addresses `alice` and `bob` exchanged 10,000 BUCKs.

8.5 Step 4: Alice Denominates 125 BUCK as Notes

Separately, Alice wants to move some of her BUCKs off the public-ledger transfer path into private notes. Her wallet picks flavour A2 and values [100 BUCK, 25 BUCK] (`totalFace = 125`), draws fresh `rho[i]` and `idHash[i]` randomness, and builds the circuit witness. `snarkjs groth16 fullProve` against `build/snark/mint/mint.wasm` and `mint_final.zkey` produces a proof triple (`pA`, `pB`, `pC`) which the wallet abi-encodes as `(uint256[2], uint256[2][2], uint256[2])` into `proofBytes`.

On-chain, Alice makes two calls:

1. `BUCK.approve(notes, 125e18, E_notes, pi_CP_notes)` – standard identity-aware approve. The Notes contract was registered `setSystemPublic(true)` by governance, so this branch takes the public-claim path (no CP proof is actually checked against Notes, though the ERC-20 allowance is still written).
2. `Notes.mint(proofBytes, [cm[0], cm[1]], 125e18)` – the `MintVerifierAdapter` forwards to the Groth16 verifier (~475K gas total); on success, 125 BUCK moves from Alice to Notes, the two `cm[i]` are appended to `commitments[]`, and `noteFaceSum + 125e18 =`. Alice’s wallet keeps the openings (`flavor`, `v`, `rho`, `idHash`, `predicate`) privately – those are the `NoteOpening` records she will need for the eventual Phase 3 `spend`.

Later, HomeCo reappraises the structure at \$280K (updated `faceValue`). Alice’s credit limit drops. If her outstanding balance exceeds the new limit, the default insurance mechanism activates (or the Jubilee fund eventually releases the lien), and she can no longer mint BUCKs.

9 Gas Cost Analysis

Approximate gas costs (Ethereum mainnet, Solidity 0.8.x optimizer). The Notes row is measured from the `MintVerifierTest.test_mint_acceptedByGroth16Verifier` Foundry run against the real Groth16 verifier; the rest are architectural estimates for the eventual PID build.

Operation	Gas (approx)	Cost at 30 gwei	Notes
BUCK_CREDIT.activate()	~50K	~\$4	Single SSTORE + event
BUCK_CREDIT.updateCredit()	~80K	~\$6	Multiple SSTOREs (insurer pays)
IdentityRegistry.register()	~235K	~\$19	PS pairing + NIZK (one-time)
IdentityRegistry.verifyApprove	~29K	~\$2	CP proof, per counterparty pair
BUCK.approve() (identity)	~97K	~\$8	29K CP + 46K allowance + 22K SSTORE
BUCK.transfer / transferFrom	~80K	~\$6	ERC-20 baseline + receipt event
BUCK.mint() (static BUCK_K)	~150K	~\$12	Enumerate credits + mint
BUCK.mint() (with PID update)	~350K	~\$28	+ Oracle reads + PID computation
Notes.mint() (Groth16)	~475K	~\$38	Groth16 verify + cm append + xfer
BUCK_CREDIT.currentValue()	0 (view)	Free	Pure computation, no state change
BUCK_K.currentBuckK()	0 (view)	Free	Read cached value

The PID update cost (~200K gas additional) is amortized across all mints within a `dT` window. With a 1-hour `dT`, only the first mint after each hour pays for the oracle reads and PID computation.

For lower gas costs, the same contracts can be deployed on Ethereum L2s (Arbitrum, Optimism, Base) where gas is 10-100x cheaper, or on alternative EVM chains. The contract architecture is chain-agnostic.

10 Security Considerations

10.1 Oracle Manipulation

- **TWAP resistance:** The BUCK/USDC price uses a Uniswap V3 TWAP (30+ minutes), requiring sustained capital commitment to manipulate.
- **Chainlink feeds:** Commodity prices from Chainlink’s decentralized oracle network, with built-in deviation thresholds and heartbeat checks.
- **Staleness checks:** `_getBasketCost()` should verify `updatedAt` is within an acceptable window (e.g., 1 hour) and revert if stale.

10.2 Credit Enumeration DoS

A malicious user could create many tiny BUCK_CREDITS to make `totalCurrentValue()` gas-prohibitive. Mitigations:

- Minimum face value for BUCK_CREDIT creation.
- Cap on credits per account (e.g., 20).
- Off-chain aggregation with Merkle proof (advanced).

10.3 Insurer Trust

The insurer can reduce `faceValue` at any time, potentially pushing accounts into default. This is by design – it reflects real-world reappraisal risk. Mitigations:

- Insurance pool covers default events.
- Multiple competing insurers provide market discipline.

- Governance can blacklist abusive insurers.
- Time-locks on insurer updates (e.g., 7-day delay for reductions > 20%).

10.4 Reentrancy

Standard OpenZeppelin `ReentrancyGuard` on `mint()` and `activate()`. All state changes occur before external calls.

11 Future Approaches

This ERC-721 implementation prioritizes clarity and correctness. Subsequent documents will explore:

11.1 ERC-1155: Semi-Fungible Batch Credits

If standardized credit classes emerge (e.g., "Calgary residential, HomeCo, LINEAR 200bp"), multiple properties in the same class could share a token ID with individual amounts. Benefits:

- Batch operations (`safeBatchTransferFrom`) for portfolio management.
- Reduced gas for accounts with many same-class credits.
- Natural fit for insurance tranches within a single credit.

Challenges: per-token depreciation start dates require additional state mapping, partially negating the fungibility benefit.

11.2 ERC-3643 (T-REX): Regulated Security Token

For production deployment under regulatory oversight:

- **ONCHAINID**: Native identity registry for KYC/AML compliance.
- **Compliance modules**: Programmable transfer restrictions (e.g., credits can only transfer to verified Alberta residents).
- **Agent roles**: Insurer as agent with granular permissions (update, freeze, force-transfer).
- **Recovery**: Lost-key recovery through identity verification.

This is the likely production standard if Alberta implements BUCK under provincial regulation.

11.3 ERC-4626: Vault-Based Insurance Pool

While a poor fit for BUCK_CREDIT itself, ERC-4626 is well-suited for the `InsurancePool`:

- Premium deposits as `deposit()` / mutual insurance shares as `shares`.
- `convertToAssets()` / `convertToShares()` for transparent share pricing.
- Standard DeFi composability (yield aggregators, lending protocols).

11.4 L2 and Cross-Chain Deployment

The contracts are EVM-compatible and can deploy on Arbitrum, Optimism, Base, or Polygon with minimal changes. Cross-chain BUCK transfers via canonical bridges or LayerZero/Axelar would enable multi-chain liquidity while maintaining a single BUCK_K oracle on the settlement layer.

12 Implementation Plan

This document specifies the **eventual** shape of the system. The first deployment ships in narrow phases so the cryptographic identity layer – the single largest piece of new ground in the design – can land first, against minimal stubs of the surrounding contracts. Each phase ends in a self-contained, deployable state with passing tests. The status table below reflects the repo at the header date; the phase subsections document the intent of each phase as originally scoped and are annotated with what the current code actually ships.

12.1 Phase Completion Status (current)

Phase	Scope	Status
Phase 0	Stub <code>BuckCredit</code> + <code>BuckKControllerStatic</code>	done – <code>BuckKControllerStatic.sol</code> live
Phase 1	<code>BN254.sol</code> helper library	done – <code>ecAdd/ecMul/ecPairing</code> wrappers + FS helper
Phase 2	<code>IdentityRegistry.sol</code>	done – PS + NIZK + CP all on-chain
Phase 3	Identity-aware <code>Buck.sol</code>	done – CP-bound approve, bilateral transfer, receipt
Phase 4	Public-claim accounts	done – <code>setPublic</code> / <code>setSystemPublic</code> ; Notes wired a
Phase 5	Notes Phase-1 scaffolding	done – <code>Notes.sol</code> + <code>StubMintVerifier</code> ; commitmen
Phase 6	Groth16 mint circuit + adapter	done – <code>circom mint.circom</code> (N=2), <code>MintGroth16Ver</code>
Phase 7	Notes Phase-3 spend circuit + nullifier reveal	pending – see Notes Phase 3
Phase 8	Live <code>BuckKController</code> (PID + oracle)	pending – contract skeleton exists; oracle hookup and
Phase 9	Production MPC trusted-setup ceremony	pending – <code>scripts/snark/setup.sh</code> uses dev-only e

Current forge suite: 100 local-EVM unit tests pass; one `BuckKControllerForkTest` requires a live mainnet RPC and is skipped offline.

12.2 Testing Strategy: Three Layers

Tests are stratified by what they actually need from the EVM, so the cryptographic and accounting work can iterate without ever touching a network:

1. **Local-EVM unit tests (default).** Foundry’s in-memory EVM ships the BN254 precompiles (`ecAdd 0x06`, `ecMul 0x07`, `ecPairing 0x08`) and all the EIP-2929 / cancun behavior we need. Every contract in this document – BUCK_CREDIT, BUCK, IdentityRegistry, the BN254 helper, the static BUCK_K – is fully exercisable without RPC, on a clean state, in milliseconds per test.
2. **Pinned mainnet/Sepolia forks (only the eventual PID BUCK_K).** When the production `BuckKController` ships and needs to read Chainlink price feeds and the Uniswap V3 BUCK/USDC TWAP, run the relevant tests with `--fork-url ${RPC} --fork-block-number ${N}`. Pinning the block number is the determinism knob: the EVM state at block *N* is immutable, and Foundry caches the RPC responses under `~/.foundry/cache/rpc/` – after the

first run the suite is offline and bit-reproducible. `vm.mockCall` covers pathological oracle paths (stale feeds, manipulated TWAPs) without needing to find a block at which they actually occurred.

3. **Differential parity with the Python reference.** The cryptographic surface is described by an executable Python reference (see **Off-Chain Wallet** below). CI regenerates the canonical JSON test vectors from that reference and `git diff --exit-code`s them against committed copies; the Solidity tests load the same vectors via `vm.readFile` and assert equality. Any drift between the spec, the wallet, or the Solidity tells us *immediately* which of the three is wrong.

The deliberate exclusion: no `vm.ffi` shelling out to Python at test time. That trades determinism, speed, and CI portability for a coupling we don't need – the wallet emits vectors once, the Solidity reads them many times.

12.3 Off-Chain Wallet (`alberta_buck.wallet`, Python)

The off-chain components – KYC ceremony, credential derivation, Chaum-Pedersen proving, and (later) SNARK witness generation for Notes – are implemented as a Python package, `alberta_buck.wallet`, with a CLI front-end. Python is the natural choice: the existing references in Identity Example and Proofs are already Python, `py_ecc` ships a complete BN254 (alt_bn128) implementation matching Ethereum's precompiles, and the package is invoked the same way every other dev tool in this repo is (`nix develop --command python3 -m alberta_buck.wallet ...`).

The wallet has three jobs:

- **Reference implementation** of the cryptography in Identity, the place where the spec is ground-truthed.
- **Test-vector emission:** a CLI verb (`wallet emit-vectors`) writes a canonical JSON file under `test/vectors/` containing the inputs and expected outputs of every on-chain proof check. Foundry tests load this file and assert agreement with the Solidity verifier.
- **End-to-end developer surface:** a CLI verb (`wallet sign`, `wallet register`, `wallet approve`) produces the calldata an end user's wallet would submit, so integration tests (and humans) can drive the contracts the same way a real wallet would.

The wallet does **not** hold private keys for production users; it is a deterministic reference and a test driver. A production wallet would reuse the same primitives behind a different storage and UI.

12.4 Phase 0: Stub the Surrounding Contracts

Goal: **Buck** can be developed and tested against a minimal, deterministic surface for **BuckCredit** and **BuckK**. All real moving parts (depreciation curves, PID, oracles) are deferred.

- `BuckKController.sol` -> replace with `BuckKControllerStatic.sol`:
 - State: `uint256 public buckK` (18 decimals, default `1e18`).
 - Functions: `currentBuckK()` `view` returns (`uint256`), `compute()` (no-op compat shim), `setBuckK(uint256)`, `transferGovernance(address)` (both governance-only).

- Drop all PID state, Chainlink basket, and Uniswap TWAP code. The full PID implementation documented above remains the design target – it slots back in once the identity layer is stable. The design-target `BuckKController.sol` also ships (unit-tested with mocked feeds); the live-oracle fork test is gated on RPC availability.
- `BuckCredit.sol` -> keep the existing implementation, but treat only one function as a load-bearing interface for Phase 0+: `totalCurrentValue(address holder) external view returns (uint256)`. No depreciation rework, no insurer-flow churn.

Deliverables: passing Foundry tests that mint, set new BUCK_K values, and watch credit limits re-derive correctly.

12.5 Phase 1: BN254 Helper Library

Goal: a single Solidity library that wraps the three BN254 precompiles and exposes the typed operations the identity contracts need.

- `src/crypto/BN254.sol` with:
 - `G1Point`, `G2Point` structs and constants for `g1`, `g2`.
 - `add(G1Point, G1Point) -> G1Point` via `ecAdd` precompile (0x06).
 - `mul(G1Point, uint256) -> G1Point` via `ecMul` precompile (0x07).
 - `pairingProd3(...)` and `pairingProd4(...)` helpers via `ecPairing` precompile (0x08).
 - Pure-Solidity Fiat-Shamir helpers: `fsRegistration(...)` and `fsChaumPedersen(...)`, each wrapping a `keccak256` over the canonical encoding from the Identity document.
 - High-level checks: `checkPSSignature`, `checkRegistrationNIZK`, `checkChaumPedersen`.

Deliverables: differential tests against the Python reference in `alberta-buck-identity-example.org` (same curve, same encoding, same Fiat-Shamir transcript). `nix develop --command python3` drives the reference; Foundry `vm.ffi` or pre-baked vector files feed the Solidity tests.

12.6 Phase 2: IdentityRegistry Contract

Goal: ship the on-chain trust anchor exactly as specified in the IdentityRegistry section.

- `src/IdentityRegistry.sol` implementing `register`, `setPublic`, `verifyApprove`, `trustIssuer`.
- All proofs verified using BN254 helpers from Phase 1.
- Tests:
 - End-to-end registration with the Python reference vectors (positive case + the four counter-examples from `alberta-buck-identity-example.org`).
 - `register` rejects untrusted issuers.
 - `register` rejects double-registration of the same address.
 - `verifyApprove` accepts a valid Chaum-Pedersen proof and rejects every documented tampered case (wrong message, wrong randomness, wrong public key, replayed challenge).
 - Gas measurements within ~15% of the published estimates (235K register / 29K verifyApprove).

Deliverables: deployable registry, passing test suite, documented gas measurements.

12.7 Phase 3: Identity-Aware Buck ERC-20

Goal: replace the current `Buck.sol` with the identity-aware version specified in the BUCK section.

- Add `IIdentityRegistry` interface and constructor wiring.
- `mint` gates on `isVerified`.
- `approve(spender, amount, E_bob, pi_CP)` with the parameterless `approve(address, uint256)` reverting.
- `transfer` / `transferFrom` via `_identityCheckedTransfer` with bilateral checks and the `BuckTransferReceipt` event.
- `_receiptFragments` storage and `_publicReceiptHash` helper.

Tests:

- Two registered users can complete the full `approve -> transferFrom` cycle and observe a `BuckTransferReceipt` whose hashes match the off-chain ciphertexts.
- Unregistered `mint` reverts.
- `transfer` to an unregistered, non-public recipient reverts.
- `transfer` without a prior CP-bound `approve` (and a non-public recipient) reverts.
- Public-claim recipients accept transfers without an `approve` step.

12.8 Phase 4: Public-Claim Account Support

Goal: AMM pools, vaults, and treasuries can interact with BUCK without needing to generate Chaum-Pedersen proofs.

- Confirm `setPublic(true)` is invocable only by registered accounts.
- Add a small `MockAMMPool` test contract that `register=s`, `=setPublic(true)=s`, then participates in `=approve/=transferFrom` flows on both sides (taker and maker).
- Verify the receipt event uses the well-known `M` for the public side and the CP-bound ciphertext for the private side.

12.9 Phase 5: Notes Scaffolding

Goal: lay the structural groundwork for BUCK Notes – a commitment pool, a pluggable mint verifier interface, and the off-chain wallet helpers – behind a stub verifier so the end-to-end flow can be exercised without yet shipping the SNARK.

Shipped:

- `src/Notes.sol` – the contract documented in the Notes section above. State is a flat `commitments[]` append list with a `commitmentExists` duplicate index and a `nullifiers` mapping reserved for the spend path. The initial Phase 1 deployment deliberately does *not* build an incremental Poseidon Merkle tree on-chain: without a SNARK asserting root consistency at mint time, a Merkle tree would be storage-heavy dead weight. The tree comes back as a state-structure change in Phase 7 alongside the spend circuit.

- `src/IMintVerifier.sol` – the narrow `verifyMint(proof, totalFace, commitments, issuer)` interface. Decoupling `Notes` from the SNARK toolchain was the key Phase-5 invariant so that the Phase-6 verifier lands as a constructor-argument swap.
- `src/StubMintVerifier.sol` – governance-toggable boolean verifier used by all `Notes` state-machine tests.
- `alberta_buck.wallet.notes` (Python) – `NoteOpening` dataclass, flavour constants, keccak-based placeholder commitment / nullifier helpers with domain-separating tags, `id_payload_a1/a2/b1` helpers matching the expected circuit public-input layout. 13 pytest tests.
- `test/Notes.t.sol` – 16 tests including happy path, duplicate detection, zero-commitment rejection, empty batch, verifier disabled, external rejecting verifier, missing approval, missing balance, state-atomicity on transfer failure.

The Python wallet’s `note_commitment` is a keccak-based placeholder; the on-chain contract stores opaque `uint256` commitments without re-hashing, so swapping keccak for Poseidon is an off-chain wallet change only.

12.10 Phase 6: Groth16 Mint Circuit + Verifier

Goal: turn the stub into a cryptographically load-bearing verifier. The mint contract must refuse to append a batch unless a real Groth16 proof ties `totalFace` to the sum of the (flavour, value, rho, idHash, predicate) openings, and each `cm[i]` to `Poseidon(5)` of its opening.

Shipped:

- `flake.nix` adds `circom` to the dev shell and bootstraps npm dev dependencies on first entry.
- `package.json` pins `circomlib` 2.0.5, `circomlibjs` ^0.1.7, `snarkjs` 0.7.5, `ethers` ^6.13.4.
- `circuits/mint.circom` – $N = 2$ mint circuit (see `Notes` section for the signal table). 648 non-linear + 1023 linear R1CS constraints.
- `scripts/snark/setup.sh` – `circom` compile, 2^{12} powers-of-tau, groth16 phase-2 setup, export verification key + Solidity verifier (renamed `MintGroth16Verifier`). Dev-only entropy.
- `scripts/snark/prove_mint.js` – witness builder (two A2 notes, `v[100,25]e18=`), seeded LCG randomness, `snarkjs.groth16.fullProve`, pB Fp2-coordinate swap, abi-encoded `proofBytes`, and sanity-check `snarkjs.groth16.verify`.
- `src/MintGroth16Verifier.sol` – `snarkjs`-generated verifier.
- `src/MintVerifierAdapter.sol` – `MINT_BATCH_SIZE=2` adapter implementing `IMintVerifier`.
- `test/MintVerifier.t.sol` – 7 tests: happy path (real proof accepted, ~475K gas), three tamper attacks (`totalFace`, `cm[0]`, batch size) all reverting with `Notes: bad mint proof`, malformed proof-bytes revert at decode, adapter constructor rejects zero.

The circuit is pinned at $N=2$. A variable- N deployment re-templates both the circuit and the adapter’s public-signals arity; this is deliberately deferred until the Phase-7 spend circuit design pins down the eventual batch-size surface.

12.11 Phase 7 and Beyond

Still pending:

- **Phase 7:** the spend circuit, nullifier reveal, and `Notes.spend` entry point – the subject of Notes Phase 3. This is where the on-chain Poseidon Merkle tree lands (a spend proof is "some unspent note in the commitment pool" – root-consistent membership is the natural query).
- **Phase 8:** live `BuckKController` deployment. The contract is already in-tree with mock-feed unit tests; the remaining work is oracle wiring (Chainlink feed addresses, Uniswap V3 pool), governance gain-setting, and a live mainnet fork test cycle before flipping the `Buck` constructor to point at it instead of the `Static` variant.
- **Phase 9:** production MPC ceremony to replace the dev-only `scripts/snark/setup.sh` trusted-setup entropy.

12.12 Sequencing and Stop Conditions

The phases are strict prerequisites of one another: Phase $n+1$ does not begin until Phase n has a passing CI run and a Foundry gas snapshot. Implementation **STOP** conditions:

- Phase 1: a Solidity `checkChaumPedersen` disagreement with the Python reference vectors. Disagreement implies a bug in the spec or in one of the implementations – resolve before any contract that depends on it ships.
- Phase 3: `BuckTransferReceipt` hash collisions across distinct ciphertexts (would indicate an ABI-encoding bug – canonicalisation matters).
- Phase 5: any Solidity-side type that does not match the chosen SNARK verifier output (revisit the toolchain choice; do not paper over with adapter contracts).
- Phase 6: a tamper-path test that fails to revert. The adapter + Groth16 verifier must reject every documented attack on `totalFace`, `cm[i]`, batch size, and malformed proof bytes; any false-accept is a hard stop before further Notes work.
- Phase 7: any inconsistency between the Phase-6 mint-circuit commitment layout and the spend circuit's opening layout (would fork the two proof systems' views of a note, silently).